Gate L4 — French polish + staff surfaces + emails + flag-flip prep

Goal

Take the platform from "French is shippable but invisible" to "French is publicly demoable at ABQ on day one." That means: close the three English-pocket leaks named in `CLAUDE.md` (server-action error strings, `lib/husbandry` helpers, Django-side errors); translate the admin and coordinator surfaces that staff actually use; localize transactional email per the D17 design; let users self-serve their email-locale preference on `/account`; and prepare the operational artifacts (checklist, runbook, env-var docs) so Aleksei can flip the flag at ABQ with confidence and roll back in under a minute if anything breaks.

L4 ships with `NEXT_PUBLIC_FEATURE_I18N_FR` still **OFF** in production. The flip is an operational decision Aleksei makes during ABQ. L4 makes the flip safe.

Scope

In scope.

- English-pocket cleanup in three named locations (server-action returns, `frontend/lib/husbandry/`, DRF + admin error messages).
- `/dashboard/coordinator/*` route translation — every visible string under `t()`, plus model `verbose_name` / `help_text` audit on the entities that surface there.
- Django admin chrome polish: `gettext_lazy` wraps on the model fields and admin labels we control. Verify `USE_I18N=True` actually drives the admin language for staff who switch.
- Email localization: `backend/i18n/email.py::send_translated_email()` helper; `User.locale` field + migration; refactor `backend/accounts/views.py:92` (signup verify) to use the helper; ship plain-text + HTML body templates for verify.
- Account-page locale field UI on `/account` (server component reads `User.locale`, server action POSTs the change to a new `PATCH /api/v1/auth/me/` field or equivalent existing endpoint).
- Pre-flip checklist + rollback runbook + Vercel env-var documentation. Verify `<LocaleSwitcher />` only renders French once `..._FR=true`.

Out of scope (do not creep).

- German UI / German content — L5.
- Spanish UI / Spanish content — L6.
- Password-reset email — the endpoint doesn't exist yet (architecture §6). When it lands, it uses the L4 helper out of the box. Do not pre-build the password-reset flow here.
- Coordinator-notification email (transfer proposed, breeding event recorded) — also not yet wired. Same disposition: helper is ready when those features ship.
- Re-running L3 MT or rebuilding the L3 review pipeline. L4 does not retranslate the corpus; L4 does not touch `backend/i18n/management/commands/translate_species.py` or the MT scripts.
- Re-translating species long-form prose past what L3 already produced. If the conservation-writer agent surfaces glossary fixes during L4, they go through the operational corrections workflow (see `docs/handover/i18n-corrections-workflow.md`), not into this gate.
- Field-level data translation in admin (translating actual Species rows) — that's L3 review-screen work and is operational, not a gate deliverable.
- Translating field-level data on Conservation Coordinator entities (institution names, program names) — out of scope; institution/program names stay as-entered.

- Performance work for `force-dynamic` × locale render cost (architecture §13.1–13.2 monitor item). L4 monitors; doesn't fix unless something regresses.

Decision lock — three English pockets close differently

CLAUDE.md i18n rule #3 names three pockets. Each closes via a different mechanism, and the choice matters because picking the wrong one creates more drift than the leak itself.

1. **Server-action error strings.** Server actions run on the server and have access to next-intl's `getTranslations()`. **Decision: translate server-side via `getTranslations()` at the server-action boundary.** Do not invent a symbolic-token contract that the client re-translates — that doubles the surface area. The action returns a localized `{ ok: false, error: string }`; the client renders it as-is.

2. **`frontend/lib/husbandry/` helpers.** These are pure TS modules that don't get a `useTranslations` hook because they may be called from non-React code paths (validation logic, formatters). **Decision: helpers return symbolic enum tokens; callers map to translated strings via a thin client-side resolver.** Pattern:

   ```
   // helper returns:
   { kind: "validation_error", code: "schooling_size_below_min", params: { min:
   6 } }
   // caller:
   const t = useTranslations("husbandry.errors");
   t(result.code, result.params);
   ```

   This is the correct trade-off because the helpers are reused across server / client boundaries; symbolic-token-with-client-resolver is the only pattern that works in both.

3. **Django-side error messages (DRF validation, admin labels). Decision: wrap with `gettext_lazy` and rely on `LocaleMiddleware`** (already shipped in L1) to honor the request's `Accept-Language`. The frontend's `apiFetch` already threads `Accept-Language` per L1 architecture §9 R6, so DRF returns French validation errors when the page is rendering under `/fr/`. Compile message catalogs via `manage.py makemessages` + `compilemessages`; wire into CI.

These three decisions are the architecture lock for L4. The story acceptance criteria below implement them; do not relitigate at implementation time.

Stop-ship checkpoints

L4 has three natural checkpoints. Each is shippable; the L4 merge requires all three.

- **Checkpoint A — English pockets closed.** Stories S1–S4. After A, `pnpm i18n:check`, the new `i18n:lint-pockets` rg sweep, and the Django `compilemessages` step all run green; rendering `/fr/contribute/husbandry` and triggering each error path returns French strings.
- **Checkpoint B — Staff surfaces + email localized.** Stories S5–S9. After B, a French- preferring coordinator sees `/fr/dashboard/coordinator` fully translated; signup-verify emails render in the user's locale (URL locale at signup time per architecture §6); the `/account` page surfaces the locale picker.

- **Checkpoint C — Flag-flip ready.** Stories S10–S12. After C, the pre-flip checklist passes against staging, the rollback runbook is in `OPERATIONS.md`, and the locale switcher correctly hides French when `..._FR=false`.

Stories

S1 — Survey the three English pockets

Audience: Implementers of S2–S4. **Checkpoint:** A. **Scope:** Discovery / docs. **Complexity:** S.

As an implementer, I want a concrete list of every leak in the three named pockets so that I can size S2–S4 accurately and so that the test-writer agent has a closed verification set.

- Run `rg` sweeps against the three pockets and produce a closed list:
  - **Server actions:** `frontend/app/**/actions.ts`, `frontend/app/**/actions/*.ts`, plus any inline `"use server"` blocks. Grep for hardcoded English in `return { ... error: ... }` and `throw new Error(...)`.
  - **`frontend/lib/husbandry/`:** every `.ts` under that directory; any function that returns a string visible to the user.
  - **Django:** DRF serializer `ValidationError("...")`, `serializers.CharField(error_messages={...})`, admin `verbose_name=`, `help_text=`, `short_description` decorators, `messages.success/error/warning(...)`.
- Output: a checklist file `docs/planning/i18n/L4-pocket-survey.md` with file:line entries grouped by pocket. This file exists for L4 only and is deleted at gate close (or moved into the PR description).
- No code changes in this story.

Acceptance:

- The survey file lists every hardcoded English string in the three pockets, by file path and line number.
- The list is bounded — no "etc." or "and similar." S2/S3/S4 implementers can tick items off.
- Survey is reviewed by the conservation-writer agent for any strings that should be folded into existing keys rather than getting new keys (avoids catalog bloat).

Dependencies: None. Day 1 work. **Test approach:** Manual review only. The survey is the artifact.

S2 — Server-action error strings localize via `getTranslations()`

Audience: Researchers and coordinators submitting via husbandry contribute form, account edit forms, any server action that surfaces an error. **Checkpoint:** A. **Scope:** Frontend (server components + new catalog keys). **Complexity:** M.

As a French-preferring contributor, when my husbandry submission fails validation server-side, I want the error message in French so that I can fix the input without falling back to English mid-flow.

- For each server action identified in S1, switch to the `getTranslations()` pattern:

```
"use server";
import { getTranslations } from "next-intl/server";
export async function submitHusbandry(...) {
  const t = await getTranslations("husbandry.contribute.errors");
```

```
  if (...) return { ok: false, error: t("invalid_locality") };
}
```

- Add the new keys to `frontend/messages/en.json` under `husbandry.contribute.errors`, `account.errors`, and any other namespaces touched. Add byte-identical placeholders to `fr.json` (will be translated at the end of S2 via the L3 pipeline or by hand for the small set of new keys), `de.json`, `es.json`. `pnpm i18n:check` enforces parity.
- Translate the new French keys via `pnpm i18n:translate fr` (scoped to the new keys) and run them through the conservation-writer voice review per the operational workflow.
- Server actions that already get a request locale (because they're called from `[locale]` pages) thread that locale through. Server actions called from API routes without locale context fall back to `getTranslations()` — which reads the active locale from the request context next-intl propagates.

Acceptance:

- Every server action in the S1 survey returns localized strings; verified by triggering each error path under `/fr/` and asserting the response content is French.
- Catalog parity check (`pnpm i18n:check`) passes after the new keys land.
- A new CI lint step `pnpm i18n:lint-pockets` runs `rg` against `frontend/app/**/actions*.ts` with a banned-pattern list (English error sentences) and fails if it finds any. Pattern list lives in `frontend/scripts/i18n-pockets.config.json`.

Dependencies: S1 (survey). **Test approach:** Vitest unit tests for one server action per namespace, asserting that `getTranslations` is called with the right namespace and that the returned error matches the French catalog value. Playwright for one happy-path flow under `/fr/contribute/husbandry` that triggers a validation error and asserts the French copy renders.

S3 — `lib/husbandry` helpers return symbolic tokens

Audience: Same users as S2. **Checkpoint:** A. **Scope:** Frontend (lib + caller updates). **Complexity:** M.

As a developer, I want `lib/husbandry` helpers to return enum-shaped errors so that callers on either side of the server/client boundary can render localized copy without the helper needing translation context.

- Refactor every user-string-returning function in `frontend/lib/husbandry/` to return the shape:

  ```
  type HusbandryResult<T> =
    | { ok: true; value: T }
    | { ok: false; code: string; params?: Record<string, string | number> };
  ```

  where `code` is a stable enum-like string (e.g., `"schooling_size_below_min"`) and `params` is the ICU-MessageFormat parameter set.
- Add a thin client-side resolver:

  ```
  // frontend/lib/husbandry/messages.ts
  export function useHusbandryMessage() {
    const t = useTranslations("husbandry.errors");
    return (result: HusbandryResult<unknown>) =>
      result.ok ? null : t(result.code, result.params ?? {});
  }
  ```

- Add server-side equivalent `getHusbandryMessage(locale)` for SSR callers.
- Add the `code` enum to `husbandry.errors.*` keys in `en.json` (and parity sibs).
- Update every caller (`frontend/app/contribute/husbandry/...` , any component or page that consumes these helpers) to use the resolver.

Acceptance:

- No `lib/husbandry/*.ts` function returns a hardcoded user-visible English string. (Verified by `pnpm i18n:lint-pockets` extending its rg sweep to this directory.)
- Existing unit tests for helpers continue to pass against the new return shape (with their expectations updated).
- Visiting `/fr/contribute/husbandry` and triggering a helper-driven validation error renders the French version.
- The `code` enum is exhaustive — TS compiler errors when a new code is added without a catalog key (via a const-asserted union of catalog keys).

Dependencies: S1, S2 (catalog wiring pattern is the same). **Test approach:** Vitest unit tests for each helper covering the refactored shape; one component test (Vitest + React Testing Library) per consumer to verify the resolver wires the right namespace; one Playwright flow under `/fr/contribute/husbandry` for end-to-end.

S4 — Django-side errors localize via `gettext_lazy`

Audience: Researchers/coordinators hitting DRF validation; admin staff seeing model labels in their preferred language. **Checkpoint:** A. **Scope:** Backend (DRF + admin). **Complexity:** M.

As a French-preferring API consumer, when DRF rejects my input, I want the validation error message in French so that the frontend can render it as-is without re-translation.

- For every entry in S1's Django pocket list, wrap with `gettext_lazy` (`from django.utils.translation import gettext_lazy as _`):
  - `serializers.CharField(error_messages={"required": _("This field is required.")})` etc.
  - `models.CharField(verbose_name=_("Population count"), help_text=_("Total individuals at the institution."))`
  - Admin `short_description = _("Approve selected")` , `actions = [...]` labels.
- Add the strings to the Django `LOCALE_PATHS = [BASE_DIR / "locale"]` pipeline:

  ```
  python manage.py makemessages -l fr -l de -l es
  ```

- Translate the French `.po` file (DeepL via `backend/i18n/management/commands/translate_po.py` — a sibling of `translate_species` — see implementation note below; if that command doesn't exist yet, hand-translate the small initial set, ~30–60 strings, and add the management command in this story).
- `compilemessages` runs in the Dockerfile build step (resolved decision 1, end of doc). `.po` files are source-controlled; `.mo` files go in `.gitignore` . CI runs a parity check that re-compiles and fails on `.po` syntax errors.
- Verify that DRF responses honor `Accept-Language` end-to-end: send `Accept-Language: fr` to a known-failing endpoint (e.g., POST to `/api/v1/auth/register/` with an invalid email) and assert the `errors.email[0]` content is French.

Acceptance:

- Every Django string identified in S1 is wrapped in `gettext_lazy`.
- `manage.py makemessages -l fr` produces a `.po` file with the survey strings.
- `manage.py compilemessages` runs cleanly in CI.
- Integration test: `POST /api/v1/auth/register/` with `Accept-Language: fr` and an invalid email returns a French error message body.
- Django admin loaded with `?language=fr` (or with a French-preferring user's session) renders model `verbose_name` / `help_text` in French for the entities we control. (Stock Django chrome — "Add", "Save", login form — is already translated by Django itself.)

Dependencies: S1. **Test approach:** Pytest integration tests for two DRF endpoints (one auth, one species) under `Accept-Language: fr` asserting French error strings. Manual smoke test of admin in French. The test-writer agent should add an adversarial case: `Accept-Language: ja` falls back to English (the safe default).

S5 — `/dashboard/coordinator/*` strings under `t()`

Audience: Conservation Coordinators (Tier 3+) using French. **Checkpoint:** B. **Scope:** Frontend. **Complexity:** M.

As a French-speaking conservation coordinator, I want `/fr/dashboard/coordinator` to render in French so that I can do my coordination work in my preferred language.

- Walk every page and component under `frontend/app/[locale]/dashboard/coordinator/` and `frontend/components/coordinator-dashboard/` (if the directory exists; else `frontend/components/`-level coordinator-specific components). Replace any remaining hardcoded English with `t()` calls under a `dashboard.coordinator.*` namespace.
- Add the new keys to `en.json`; placeholder + translate to `fr.json`; placeholder copies to `de.json` / `es.json` (those translate in L5/L6).
- Verify the dashboard data fetcher (`frontend/lib/coordinatorDashboard.ts`) threads the active locale into its `apiFetch` calls so backend-served labels (e.g., breeding-status enum displays) come back localized.

Acceptance:

- `pnpm i18n:check` passes.
- The dashboard's `i18n:lint-pockets` sweep (extended to include `app/[locale]/dashboard/` and any coordinator-specific component dirs) finds no hardcoded English.
- Manual smoke: log in as a Tier 3 user, visit `/fr/dashboard/coordinator`, confirm headings, table column labels, status pills, action buttons are all French. Confirm the cache discipline still holds (no leakage between locales) by hitting the page first under `/fr/` then under `/` and asserting different rendered HTML.

Dependencies: S2 (server-action localization sets the catalog pattern). Can run in parallel with S3, S4. **Test approach:** Vitest component tests for two representative coordinator-dashboard components under `/fr/`. Playwright session-driven test that logs in, visits the dashboard under each locale, asserts visible-text French/English deltas.

S6 — Django admin labels localized

Audience: Tier 5 admin operators who prefer French. **Checkpoint:** B. **Scope:** Backend. **Complexity:** S.

As a French-preferring admin, I want the Django admin to render the labels we control in French so that the admin UI is consistent with the rest of the platform.

- Audit `backend/*/admin.py` for hardcoded English strings: `list_display` callable `short_description`, `list_filter` titles, `actions` `short_description`, custom `verbose_name_plural`, fieldset names. Wrap with `gettext_lazy`.
- The model-level `verbose_name` / `help_text` audit happens in S4; this story is the admin-class layer specifically.
- Verify the language picker in admin works: a staff user with `User.locale="fr"` (S7) lands in French admin; a staff user with no preference set falls back to `Accept-Language`.

Acceptance:

- Admin classes for `Species`, `Taxon`, `ConservationAssessment`, `Institution`, `ExSituPopulation`, `BreedingEvent`, `Transfer`, `BreedingRecommendation`, `User`, `TranslationStatus` have all their custom labels wrapped in `gettext_lazy`.
- The S4 `.po` file picks up the new strings on `makemessages`.
- Smoke test: an admin user with `locale="fr"` sees French labels for the entities we control.

Dependencies: S4 (shares the `.po` pipeline). **Test approach:** Pytest test that loads the admin index with a French-locale staff user and asserts French copy in known label slots. Manual visual smoke of two admin entity pages.

S7 — `User.locale` field + migration

Audience: All users (used by emails in S8 and the account UI in S9). **Checkpoint:** B. **Scope:** Backend. **Complexity:** S.

As an authenticated user, I want my preferred locale stored against my account so that the platform can send me emails and (in future) default-render the UI in my language.

- Add `locale` field to `backend/accounts/models.py`:

```
locale = models.CharField(
    max_length=5,
    choices=settings.LANGUAGES,
    default="en",
    help_text=_(
        "Preferred locale for transactional emails and (when logged in) "
        "the default UI locale on first visit. Auto-set from the locale "
        "the user is signed up under; user-changeable from the account page."
    ),
)
```

- Migration: `AddField` with `default="en"`. Backfills every existing user with English. Safe — matches their current behavior.
- Expose `locale` on `GET /api/v1/auth/me/` (read).
- Add `PATCH /api/v1/auth/me/` (or the existing endpoint if it already supports patch — verify during implementation; if it doesn't, the cleanest add is a small `MeUpdateView`) accepting `{ "locale": "fr" }`. Validates against `settings.LANGUAGES`. Returns the updated user payload.
- Update `backend/accounts/views.py` register flow to set `locale` from the request's active language at signup time, so a user who signs up under `/fr/signup` gets `locale="fr"` on day one. (Architecture §6 explicitly addresses this; the helper in S8 falls back to `settings.LANGUAGE_CODE` if the field is empty.)

Acceptance:

- Migration applies cleanly on a fresh DB and on staging (existing users get `locale="en"`).
- `GET /api/v1/auth/me/` returns the `locale` field.
- `PATCH /api/v1/auth/me/ {"locale": "fr"}` succeeds; subsequent `GET` reflects the change.
- Invalid `locale` values (`"ja"`, `""`, `"FR"` uppercase) return 400 with a localized error.
- Signing up via `/fr/signup` results in a `User` row with `locale="fr"` after verify completes.

Dependencies: None on the backend side; runs in parallel with S5/S6. **Test approach:** Pytest model migration test (apply on a fixture DB with one existing user, assert the user's `locale` is `"en"`); pytest API tests for `GET` and `PATCH` including the validation cases.

S8 — `send_translated_email()` helper + verify-email refactor

Audience: New signups (right now); password-reset and coordinator-notification users (future, when those features ship). **Checkpoint:** B. **Scope:** Backend. **Complexity:** M.

As a new signup, I want my verification email in my preferred locale so that the link copy and instructions are readable without falling back to English.

- Implement `backend/i18n/email.py::send_translated_email()` per the architecture spec §6 signature. Resolution order: explicit `locale=` kwarg → `recipient.locale` (S7) → `settings.LANGUAGE_CODE`.
- Render three templates per email:
  - `{template}_subject.txt` — single-line subject.
  - `{template}_body.txt` — plain-text body (always sent).
  - `{template}_body.html` — HTML body (optional; if missing, plain-only).
- One template file per name; `{% trans %}` / `{% blocktranslate %}` provide the localizable strings. `translation.override(locale)` wraps the render. **Do not** suffix templates per locale. The `.po` catalog from S4 is the single source.
- Refactor `backend/accounts/views.py:92` to call the helper:

```
send_translated_email(
    recipient=user,
    template="accounts/verify_email",
    context={"verification_url": verification_url, "user": user},
    locale=request.LANGUAGE_CODE,  # locale of the signup page; falls back to
en.
    fail_silently=True,
)
```

- Create the three template files at `backend/accounts/templates/accounts/`:
  - `verify_email_subject.txt` — was inline; move to template with `{% trans %}`.
  - `verify_email_body.txt` — was inline plain text; move to template.
  - `verify_email_body.html` — **new for L4**. A minimal, accessible, brand-aligned HTML email. Coordinate copy with the conservation-writer agent.
- Run `makemessages` again to pick up the template strings; translate French; commit.

Acceptance:

- `send_translated_email()` exists and accepts the documented signature.
- A unit test calls it with `recipient=<user with locale="fr">` and asserts the rendered subject / body match the French catalog.

- A unit test calls it with explicit `locale="de"` overriding a `locale="fr"` user and asserts the German render (German catalog will be a placeholder copy of English in L4 — that's fine; the test asserts the `translation.override("de")` was invoked).
- Signup integration test: register via `/fr/signup`, assert the verify email's subject/body content is French (use Django's `mail.outbox` test fixture).
- HTML body is sent as `multipart/alternative` with both `text/plain` and `text/html` parts.
- The architecture's "no notifications app yet — refactor when there's a third channel" note is captured as a comment in `backend/i18n/email.py`.

Dependencies: S4 (`.po` pipeline), S7 (`User.locale`). **Test approach:** Pytest unit tests for the helper (resolution order, template rendering under override, fail-silently behavior). Pytest integration test for the signup flow under `/fr/signup` asserting the outbox content.

S9 — Locale picker UI on `/account`

Audience: Authenticated users. **Checkpoint:** B. **Scope:** Frontend (server component + server action). **Complexity:** S.

As a French-preferring user who signed up under `/`, I want to set my email locale on my account page so that future verification, notification, and password-reset emails reach me in French.

- Add a locale `<select>` to `frontend/app/[locale]/account/page.tsx` showing the four locale labels in their own scripts (English, Français, Deutsch, Español).
- Pre-populate from `getServerSession` → fetch `/auth/me/` → render `locale`.
- Add a server action `updateAccountLocaleAction` in `frontend/app/[locale]/account/actions.ts` that PATCHes `/api/v1/auth/me/` with the new locale. Use the L4 server-action localization pattern (S2): the action returns localized success/error strings via `getTranslations`.
- Show a confirmation banner ("Locale updated. Future emails will be sent in Français.") on success. Show a localized error on failure.
- The picker is **not** a UI-language switcher; that's `<LocaleSwitcher />`. Add a one-line help text under the select clarifying: "This affects emails we send you. To change the site language, use the language menu in the header." (Both strings under `t()`.)
- Per architecture §13.7, this is the L4 surface for the `User.locale` field.

Acceptance:

- `/account` renders the locale picker with the user's current preference selected.
- Changing the selection and submitting persists the change (verifiable via a follow-up `GET /me/`).
- The success and error states render in the active UI locale (so a French-rendering account page shows French success copy regardless of which locale the user picked).
- Anonymous visitors to `/account` are still redirected to login per Auth Gate 11; this story does not alter middleware.

Dependencies: S7 (`/me/` PATCH endpoint), S2 (server-action localization pattern). **Test approach:** Vitest component test for the picker rendering with mock session data. Playwright session-driven test: log in, visit `/account`, change locale, refresh, assert the selection persists. Pytest integration test for the PATCH endpoint covered in S7.

S10 — Per-locale flag verification: French only when `..._FR=true`

Audience: Operators (Aleksei) and visitors. **Checkpoint:** C. **Scope:** Frontend (regression + verification, no new code expected). **Complexity:** S.

As an operator, I want certainty that flipping `NEXT_PUBLIC_FEATURE_I18N_FR` is the *only* operational lever for making French publicly visible, so that I can flip it confidently at ABQ without redeploying.

- L1's S10 already implemented per-locale flag gating. L4 verifies it end-to-end against the post-L3 codebase:
  - With `NEXT_PUBLIC_FEATURE_I18N=true` and `NEXT_PUBLIC_FEATURE_I18N_FR=false` (current prod state): `<LocaleSwitcher />` shows English only. `/fr/...` requests return 404 (or redirect to `/`, depending on the L1 implementation — verify and document).
  - With `NEXT_PUBLIC_FEATURE_I18N_FR=true`: switcher shows English + Français; `/fr/...` routes resolve and render French content.
- Audit `frontend/components/LocaleSwitcher.tsx` and `frontend/middleware.ts` for any drift introduced by L2/L3/L4 work that would let French leak through with the flag off.
- If any drift is found, fix it in this story. Otherwise this story is verification-only.
- Add a Playwright test fixture matrix that runs the smoke suite (S11 in L1) under both flag states and asserts the expected switcher and route behavior.

Acceptance:

- With `..._FR=false`, no `<a>` element pointing at `/fr/` is in the DOM of any page; no `/fr/...` URL resolves with 200.
- With `..._FR=true`, the switcher includes Français and `/fr/species/123` returns 200 with French content (assuming L3 content is approved for that species — see pre-flip checklist).
- The matrix test is run on every PR under both flag states.

Dependencies: S5 (so the staff dashboard is included in the verification pass). **Test approach:** Playwright matrix as described. Manual smoke on staging with the flag toggled.

S11 — Pre-flip checklist + rollback runbook

Audience: Aleksei (the operator who flips the flag at ABQ). **Checkpoint:** C. **Scope:** Docs. **Complexity:** S.

As Aleksei, I want a written checklist I can run in 10 minutes before flipping the French flag at ABQ, and a rollback runbook I can execute in under 60 seconds if anything regresses during the demo, so that I'm not improvising under audience pressure.

- Create `docs/operations/i18n-french-flip-checklist.md` covering:
  - **Pre-flip checks (run on staging with `..._FR=true`):**
    - `pnpm i18n:check` and `pnpm i18n:lint-pockets` are green on `main`.
    - The L3 review queue has zero `mt_draft` rows for the demo families (Bedotiidae, Cichlidae). Verify via the admin `TranslationStatus` filter.
    - Smoke: visit `/fr/`, `/fr/species/`, `/fr/species/<demo-id>/`, `/fr/map/`, `/fr/dashboard/coordinator` (logged in). All render French; no `(English)` fallback badges on the demo species detail.
    - Sign up via `/fr/signup` with a throwaway address; verify the email arrives in French; click the link; verify the redirect-to-login flow works.
    - `<LocaleSwitcher />` shows Français; flipping back to English preserves the path.
  - **Flip step:** in Vercel production env, set `NEXT_PUBLIC_FEATURE_I18N_FR=true`. No redeploy required (it's a public env var; a redeploy is triggered automatically by Vercel on env-var change).
  - **Post-flip verification:** repeat the smoke set against production; confirm `https://malagasyfishes.org/fr/` returns 200; check that `/sitemap.xml` continues

to list the French alternates (already the case in L1).

- Create `docs/operations/i18n-french-rollback-runbook.md` covering:
  - **Symptom triage:** "site looks broken under `/fr/`," "verification email sends in English even for French signups," "switcher missing entries," "search engines crawled a half-translated page."
  - **Rollback action:** in Vercel production env, set `NEXT_PUBLIC_FEATURE_I18N_FR=false`. Locale switcher hides Français; `/fr/...` URLs revert to whatever L1 specified (likely a 404 or redirect to `/`). No code revert needed.
  - **Deeper rollback:** if `NEXT_PUBLIC_FEATURE_I18N=false` (master flag), the entire locale switcher disappears and `/fr/`, `/de/`, `/es/` all 404. This is the nuclear option; document it but note it would also surface English-only behavior elsewhere (e.g., admin staff who set `User.locale="fr"` keep seeing French in the admin because that's middleware-driven, not flag-driven).
  - **What rollback does NOT undo:** users who signed up via `/fr/signup` between flip and rollback have `User.locale="fr"` persisted; their future emails still localize. That's correct behavior, not a bug — but document it so a panicking operator doesn't go looking.
- Update `OPERATIONS.md` with a pointer to both runbooks under a new "i18n" subsection.

Acceptance:

- Both runbook files exist and are reviewed by the conservation-writer agent for clarity.
- Aleksei reviews and signs off (verbally; no formal approval gate).
- The pre-flip checklist passes against staging with `..._FR=true` set.

Dependencies: S10 (verification logic locked). **Test approach:** Manual rehearsal — Aleksei (or the implementer impersonating him) runs the checklist top-to-bottom against staging, then flips and reverts the staging flag, then runs the rollback runbook. Note any friction and fold into the doc.

S12 — `CLAUDE.md` and `OPERATIONS.md` updates; pocket-lint in CI

Audience: Future maintainers. **Checkpoint:** C. **Scope:** Docs + CI config. **Complexity:** XS.

- Update the `CLAUDE.md` "i18n (Gate L1)" section's three-pockets rule (#3) to reference the L4-shipped resolutions: server-action `getTranslations`, `lib/husbandry` symbolic tokens, Django `gettext_lazy` + `Accept-Language`. The rule changes from "L4 polish, not L1 work" to "L4 shipped these patterns; new code follows them."
- Add an "Email localization" subsection under i18n in `CLAUDE.md` pointing at `backend/i18n/email.py` and noting that future channels (password-reset, coordinator-notify) plug into the same helper.
- Add the `pnpm i18n:lint-pockets` step to CI (`.github/workflows/ci.yml` or equivalent). Document its banned-pattern config in `frontend/scripts/i18n-pockets.config.json`.
- Update `OPERATIONS.md` secrets/env section with `NEXT_PUBLIC_FEATURE_I18N_FR` rollout notes (referencing S11's runbook).

Acceptance:

- `CLAUDE.md` reflects the post-L4 reality.
- CI runs `pnpm i18n:lint-pockets` on every PR and fails on banned-pattern matches.
- `OPERATIONS.md` lists the L4 env vars alongside `NEXTAUTH_SECRET`, `COORDINATOR_API_TOKEN`, `DEEPL_API_KEY`.

Dependencies: S2, S3, S4, S5, S8, S11 (this story consolidates everything else). **Test approach:** Manual review. Verify CI runs the new lint step on a PR by intentionally inserting a banned-pattern violation and reverting after the failure is confirmed.

Sequencing / parallelization

L4 is four weeks (May 1 → May 29). The dependency DAG:

```
S1 (survey, day 1) ────────────────┬── S2 (server actions, days 2–6) ────┐
                                   ├── S3 (lib/husbandry, days 2–6) ──────┤
                                   └── S4 (Django gettext, days 2–7) ─────┤
                                                                          │
                       S7 (User.locale, days 2–4) ── S8 (email helper + refactor,
days 5–9) ──┤
                                                                          │
│
                                            S5 (coordinator dashboard, days 4–9)
────────────┤
                                            S6 (Django admin labels, days 5–8)
─────────────┤
                                                                          │
│
                                            S9 (account locale picker, days 6–9)
────────────┤
                                                                          │
│
                                                                          │
│
                                                                          ├── S10 (flag
verification, days 10–11) ─┐
                                                                          │
│
                                                                          └── S11
(runbooks, days 11–13) ──────────┤

│

S12 (docs, days 13–14)
```

Wave 1 (week 1): S1 (1 day, sequential), then S2/S3/S4/S7 in parallel. **Wave 2 (week 2):** S5, S6, S8, S9 in parallel — each consumes one of the wave-1 outputs. **Wave 3 (week 3):** S10 (verification), S11 (runbooks). **Wave 4 (week 4):** S12 (docs), buffer for surprises, security/code-quality reviewer passes, Aleksei sign-off on the runbooks.

Two implementer slots can run waves 1 and 2 in parallel (one frontend, one backend) and finish by end of week 2. Week 3–4 is verification + buffer + ABQ travel prep.

Verification gates

Before merging to main:

- `@security-reviewer` — focus on the email helper (no recipient/sender injection via locale parameter; HTML body XSS surface from rendered context vars), the `User.locale` PATCH endpoint (validation against the LANGUAGES choices, no privilege escalation), and the `Accept-Language` plumbing (does it interact with the auth-tier gate per the L1 architecture §9 R1? — verify no regression from L1).
- `@code-quality-reviewer` — focus on the three pocket closures (consistent patterns), the `User.locale` migration (default-en safety, no data loss), and the runbook clarity.
- `@test-writer` — adversarial cases listed below.

Adversarial test scenarios for the test-writer agent

- **Locale-injection in `send_translated_email` `locale=` kwarg:** pass `locale="../etc/passwd"`, `locale="<script>"`, `locale=""` — all should fall back safely to `settings.LANGUAGE_CODE` without raising or rendering attacker-controlled paths.
- **`Accept-Language: ja,en;q=0.5`:** weighted unsupported-then-supported. Should resolve to `en`, not `ja`. (Django default behavior; verify it isn't broken by middleware ordering.)
- **PATCH `/me/` with `locale="EN"` (uppercase):** must 400 — not silently coerce.
- **Signup under `/fr/signup`, then PATCH locale to `de`, then password-reset (when L5/L6 ships):** user gets German emails. Out-of-scope for L4 testing but the flow should be designed not to regress when password-reset lands.
- **HTML email render with attacker-controlled `user.name`:** assert the template escapes HTML in the user's display name. (Django's template engine does this by default; verify no `|safe` filter slips into the templates.)
- **Cache contamination:** load `/fr/dashboard/coordinator` as a French-preferring Tier 3 user, immediately load `/dashboard/coordinator` as an English-preferring Tier 3 user; assert each gets their expected locale's labels. (L1 already covers this for public pages; L4 extends to authenticated pages.)
- **Flag-flip race:** user signs up under `/fr/signup` while `..._FR=true`, then operator flips `..._FR=false` mid-verification. The verify URL points at `/verify` (English path) per L1's localized-redirect rule; verify the user can still complete signup. Confirm `User.locale="fr"` persists for the user's future emails.

Definition of done

- All twelve stories complete with their acceptance criteria green.
- CI green: `pnpm i18n:check`, `pnpm i18n:lint-pockets`, `compilemessages`, the existing test matrix, and the new flag-state Playwright matrix.
- French is fully demoable on staging with `..._FR=true`. The pre-flip checklist passes.
- The rollback runbook is reviewed and signed off by Aleksei.
- `gate/L4-i18n-french-staff` merged to `main` behind `NEXT_PUBLIC_FEATURE_I18N_FR=false` in production.
- `docs/planning/i18n/README.md` Gate L4 row updated to ✅.

Risks and mitigations

- **R1 — Translation review queue blocks the flip.** L4 doesn't retranslate the corpus, but any new keys added in S2/S3/S5 need French translations through the conservation-writer agent and human approval. **Mitigation:** keep new-key counts bounded (each story's AC caps the catalog growth); batch the new-key review at the end of week 2 so Aleksei has one review pass, not five.
- **R2 — `gettext_lazy` import-order bugs.** `_("...")` at module-import time before Django is configured raises `ImproperlyConfigured`. **Mitigation:** all `_()` calls must be inside classes or

functions, never at module top level. Code-quality reviewer to verify.

- **R3 — `User.locale` migration on prod with existing users.** The migration is `AddField` with a default, which Django backfills in a single statement. On the current row count (handful of users) this is instant. **Mitigation:** none needed at this scale, but document in the migration's docstring that future scale would warrant a multi-step migration.
- **R4 — HTML email render breaks deliverability.** A new HTML body template can trigger spam filters if it's not aligned with text body content or has broken markup. **Mitigation:** lint the HTML through a litmus-style checker before staging; Aleksei forwards a copy to Gmail and Outlook from staging during the S11 checklist rehearsal.
- **R5 — Pocket survey misses a leak.** A hardcoded English string survives into post-L4 prod, surfaces during ABQ. **Mitigation:** the `i18n:lint-pockets` CI rule is the defense-in-depth; the manual smoke in S10 is the second layer; the runbook in S11 acknowledges the pattern and provides a same-PR fix path (add the string to en.json, translate, redeploy).
- **R6 — `Accept-Language` × auth-token interaction regresses.** L1 architecture §9 R1 warned about this. **Mitigation:** S5 explicitly tests the cache-contamination case for authenticated routes; the test-writer adversarial scenarios extend it.

What's deliberately deferred

These are real items that L4 deliberately does not ship. Each has a clear successor.

- **German UI (L5).** All `de.json` keys ship as English placeholders out of L4. L5 owns the German translation pass through the conservation-writer voice review.
- **Spanish UI (L6).** Same disposition for `es.json`.
- **Full-corpus species translations beyond L3 output.** L3 already translated the species long-form prose to French; L4 doesn't retranslate. If glossary fixes surface during L4, they go through the operational corrections workflow (`docs/handover/i18n-corrections-workflow.md`).
- **Password-reset email (no successor gate; whenever the feature ships).** The endpoint doesn't exist yet (architecture §6). When it lands, the implementer calls `send_translated_email(template="accounts/reset_password", ...)` and adds the three template files. No L4 work prefigures this.
- **Coordinator-notification email (no successor gate; whenever the feature ships).** Same disposition. Architecture §6 explicitly says "no notifications app yet — refactor when there's a third channel."
- **Localized search across translatable fields (L3+ parking-lot item).** A French speaker searching "rouge" should match `description_fr`. Architecture §12 Q-NEW2. Scope is a UX call about rank stability across locales; L4 doesn't have the bandwidth.
- **`force-dynamic` × locale render cost monitoring (architecture §13.1–13.2).** L4 monitors via the Vercel analytics dashboard during the ABQ window; doesn't act unless something regresses. If perf is an issue, the architecture doc identifies the fix (push heavy fetches into route handlers that cache per locale).
- **L3 review-pipeline rebuild.** The pipeline shipped in L3 (`backend/i18n/management/commands/translate_species.py`, the admin review screen, the `TranslationActualLocaleMixin` gating). L4 consumes it; doesn't modify it.
- **Translation of `Institution.name`, `CoordinatedProgram.title`, etc.** Field-level data on coordinator entities stays as-entered. If a French-language program needs a French title in admin, that's a content-entry concern, not a localization concern.
- **`L4 monitor` for `force-dynamic` /locale render cost** — see above; deferred to ABQ-time observation.

Resolved decisions (locked 2026-04-30)

1. **`.mo` compilation runs in the Dockerfile build step.** `python manage.py compilemessages` runs once at image build, baked into the deployed artifact. Reasons: same `.mo` files across dev/staging/prod, no runtime `gettext` dependency, no per-deploy compile cost. `.po` files are source-controlled; `.mo` files are added to `.gitignore` (build artifact, like `.pyc`). The Dockerfile already has `gettext` indirectly via `apt-get` toolchain; add explicit `gettext` to the apt list if it isn't already covered. CI keeps a parity check that re-compiles in a throwaway container and asserts the build succeeds — catches `.po` syntax errors before deploy.

2. **HTML email template scope: full branded kit.** S8 ships:

   - A shared base template at `backend/templates/email/base.html` carrying:
     - Header band with platform logo + name (links to canonical URL)
     - Body region with localized greeting + content blocks
     - Footer with platform contact (`alex.saunders@wildlifeprotectionsolutions.org`), project URL, and unsubscribe-equivalent text (account → notification preferences link, even though notification preferences are L5+ — link can land on /account in L4 and route to the future preferences sub-page)
   - The verify-email template (`accounts/verify_email_body.html`) extends the base.
   - A plain-text twin (`_body.txt`) is required for every HTML email (deliverability, accessibility, clients that block HTML).
   - Branding assets (logo) inlined as `<img>` referencing absolute URLs on the deployed site rather than embedded as data-URIs (keeps email size down, fine for transactional mail since the recipient just signed up and is likely to load images anyway).
   - Color palette + typography pulled from the site's design tokens; reasonable email-safe fallbacks (Helvetica/Arial) layered behind Spectral/IBM Plex Sans. Email clients don't reliably load web fonts.

   This commits us to a slightly larger S8 (template work + asset paths) but produces an email that's demo-ready for ABQ.